



1. 线性表是一种由 n 个数据元素组成的数据结构, n 的取值是
A. 0或者任意一个正整数或者 ∞
B. 非负整数
C. 任意一个正整数或者 ∞
D. 某个正整数
2. 在一个单链表中,已知 q 所指结点是 p 所指结点的后继结点,若在 p 和 q 之间插入 s 所指结点,则正确的操作是
A. $s \rightarrow \text{next} = p \rightarrow \text{next}; p \rightarrow \text{next} = s;$
B. $s \rightarrow \text{next} = q; p \rightarrow \text{next} = s \rightarrow \text{next};$
C. $q \rightarrow \text{next} = s; s \rightarrow \text{next} = p;$
D. $p \rightarrow \text{next} = s; s \rightarrow \text{next} = p;$
3. 下列选项中,不宜通过栈求解的问题是
A. 判断字符串是否是回文
B. 检验圆括号是否匹配
C. 不同数制之间进行转换
D. 图的广度优先搜索遍历
4. 设栈 S 的输入序列为1,2,3,4,5,则下列选项中不可能是 S 的输出序列的是
A. 2,3,4,1,5
B. 5,4,1,3,2
C. 2,3,1,4,5
D. 1,5,4,3,2
5. 使用一个大小为6的数组保存循环队列 Q 。若从 Q 中出队两个元素,并入队一个元素,此时队尾 rear 和队头 front 的值分别为2和4。则在执行这三个操作之前 rear 和 front 的值分别是
A. 0和3
B. 1和2
C. 2和5
D. 4和5
6. 设二维数组 M 有3行4列,按行优先的方式存储,每个元素占6个存储单元。第1个元素的存储地址为100,则 $M[2][2]$ 的存储地址为A
A. 135
B. 153
C. 160
D. 165
7. 设 n 阶方阵 M 是对称矩阵,采用压缩存储方式将 M 中的元素保存在一维数组 B 中,则下列选项中,正确的是
A. 保存 M 中的主对角线中的元素, B 的元素个数是 n
B. 保存 M 中上三角部分的元素, B 的元素个数是 $n(n-1)/2$
C. 保存 M 中上三角部分的元素, B 的元素个数是 $n(n+1)/2$
D. 保存 M 中的全部元素, B 的元素个数是 n^2

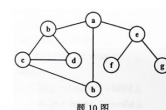
8. 已知完全二叉树T的第4层有5个叶结点, 则T的结点个数最多是

- A. 12
- B. 20
- C. 21
- D. 36

9. 在一棵非空二叉树的后序遍历序列中, 所有列在根结点前面的是

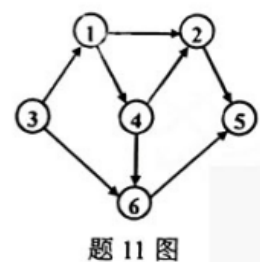
- A. 左子树中的部分结点
- B. 右子树中的全部结点
- C. 左右子树中的部分结点
- D. 左右子树中的全部结点

10. 若对题10图所示的无向图进行深度优先搜索遍历, 则下列选项中正确的遍历序列是



- A. h, c, a, b, d, e, g, f
- B. e, a, f, g, b, h, c, d
- C. d, b, c, a, h, e, f, g
- D. a, b, c, d, h, e, f, g

11. 对题11图所示的有向图进行拓扑排序。下列选项中能够得到的拓扑序列是



- A. 3, 1, 2, 4, 5, 6
- B. 3, 1, 2, 4, 6, 5
- C. 3, 1, 4, 2, 5, 6
- D. 3, 1, 4, 2, 6, 5

12. 已知数据序列(8, 9, 10, 4, 5, 6, 20, 1, 2)是某种排序算法第一趟排序后得到的结果, 则该算法可能是

- A. 选择排序
- B. 起泡排序
- C. 直接插入排序
- D. 快速排序

13. 下列选项中, 每一趟都能选出一个元素放在其最终位置上, 且不稳定的排序算法是

- A. 起泡排序
- B. 希尔排序
- C. 归并排序
- D. 快速排序

14. 对有序表(1, 9, 12, 41, 62, 77, 82, 95, 100)采用二分查找方法查找值82, 查找过程中关键字的比较次数是

- A. 1
- B. 2
- C. 4
- D. 7

15.将下列数据依次插入到初始为空的二叉排序树中能得到高度最小的二叉排序树的序列是

- A.2,4,7,5,8,10
- B.5,1,2,6,3,4
- C.6,4,1,8,10,5
- D.9,7,2,1,4,0

16.线性表的存储方式中，能够随机存取表中任一元素的存储结构是_____。

17.用S表示入栈操作，X表示出栈操作，若元素入栈顺序为1234，为了得到1342的出栈顺序，相应的S、X操作串为_____。

18.若广义表L的深度是 ∞ ，则L一定是_____。

19.广义表 $((a,b), (c,d), e)$ 的表尾是_____。

20.利用二叉树中的空指针域，使之指向结点在某种遍历次序下的前趋或后继结点，此时域中的内容称为_____。

21.若用n个带权字符构造哈夫曼树T，则T中结点的总数是_____。

22.设连通带权图G中有n个顶点，使用普里姆算法构造G的最小生成树T，T中含有的边数是_____。

23.要使n个记录的关键字序列 $k_1, k_2, \dots, k_n$ 成为小根堆，关键字之间必须满足的关系是_____。

24.索引顺序查找是一种将顺序查找和二分查找思想结合在一起的查找方法，又称为_____。

25.5阶B树T中，除根结点之外每个结点中所含关键字个数最少是_____。

26.设细数矩阵M如下所示。矩阵的行列下标均从1开始，请画出M按行优先存储的三元组表。

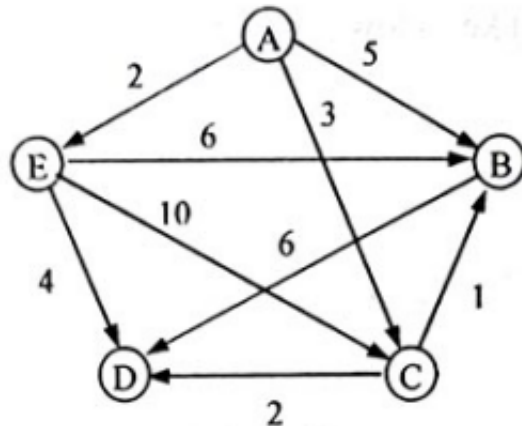
27. 已知二树T的前序遍历序列是A,B,C,D,E,L,M,O,N序遍历序列是C,B,E,D,A,M,O,L,N请画出T。

28. 已知有向带权图G如题28图所示。

请回答下列问题。

(1) 给出图G的邻接矩阵。

(2) 求出G中从源点A到其余各顶点的最短路径。要求根据迪杰斯特拉算法的求解过程依次给出各条路径,包括路径上经过的顶点及其长度。



题 28 图

29. 设有关键字序列(65,23,31,26,7,91,53,15,72,52),散列函数为 $H(\text{key})=\text{key}\%11$,将关键字依次放入表长为11的散列表H中,采用线性探测法处理冲突。请回答下列问题。(1)画出构造的散列表,并给出查找每个关键字的探查次数。(2)求散列表的平均查找长度ASL。

30. 顺序表类型定义如下:

```
# define ListSize 100
```

```
typedef struct {
```

```
    int data[ListSize];
```

```
    int length;
```

```
} SeqList;
```

阅读下列算法,并回答问题。

```
void mysum(SeqList *SL1, SeqList *SL2)
```

```
{    int minlength, k = 0;
```

```
    minlength = SL2->length;
```

```
    for ( k = 0; k < minlength; k++ )
```

```
        if ( SL1->data[k] < SL2->data[k] ) SL1->data[k] += SL2->data[k];
```

```
        else SL2->data[k] += SL1->data[k];
```

```
    return;
```

```
}
```

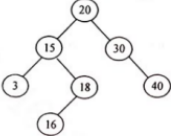
```
void f0( SeqList *SL1, SeqList *SL2 )
{
    if ( SL1->length > SL2->length ) mysum( SL1, SL2 );
    else mysum( SL2, SL1 );
    return;
}
(1) 若 SL1->data 中的数据为 (52,14,256,-9,-38,30,128,257,64), SL2->data 中的数据
    为 (32,14,-63,15,29,51,16,8), 则执行算法 f0(&SL1, &SL2)后 SL1->data 和
    SL2->data 中的数据各是什么?
(2) 该算法的功能是什么?
```

31. 31. 二叉树的存储结构类型定义如下:

```
typedef int DataType;
typedef struct node
{
    DataType data;                // data 是数据域
    struct node * lchild, * rchild; // 分别指向左右孩子
} BinTNode;
typedef BinTNode * BinTree;
阅读下列算法, 并回答问题。
int height( BinTree T )
{
    int lhigh = 0, rhigh = 0;
    if ( T == NULL ) return 0;
    else {
        lhigh = height( T->lchild );
        rhigh = height( T->rchild );
        if ( lhigh > rhigh ) return lhigh + 1;
        else return rhigh + 1;
    }
}
void f31( BinTree T )
{
    int leftHigh = 0, rightHigh = 0;
    BinTree temp;
    if ( T == NULL ) return;
```

```
else {
    if ( height( T->lchild ) < height( T->rchild ) ) {
        temp = T->lchild;
        T->lchild = T->rchild;
        T->rchild = temp;
    }
}
B1( T->lchild );
B1( T->rchild );
return;
```

(1) 设二叉树 T 如题 31 图所示，画出执行 B1(T)后得到的二叉树 T1。



题 31 图

(2) 给出函数 B1()的功能。

32. 32. 设顺序表的存储类型定义如下：

```
#define ListSize 100
typedef int KeyType;
typedef struct {
    KeyType key;
} NodeType;
typedef NodeType SeqList[ListSize];
```

函数 f32()的功能是，基于二分查找在长度为 n 的有序表 R 中插入一个关键字 x，并保持 R 的有序性。请在空白处填上适当语句使算法正确。

```
void f32( SeqList R, KeyType x, int n )
{
    int low = 0, high = n-1, mid, inspace, i, find = 0;
    while ( low <= high && !find ){
        mid = ( low + high )/2;
```

```
if ( x < R[mid].key ) (1) _____;
else if ( x > R[mid].key ) low = mid + 1;
else find = 1;
}
if ( find )
    inspace = (2) _____;
else
    inspace = low;
for ( i = n; (3) _____; i--)
    R[i+1] = R[i];
R[inspace].key = x;
}
```

33.

33. 设顺序表的存储类型定义如下:

```
typedef int KeyType;
```

```
typedef struct {
```

```
    KeyType key;
```

```
} RecType;
```

阅读下列算法, 并回答问题。

```
int f33( RecType R[], int i, int j )
```

```
{
```

```
    RecType x = R[i];
```

```
    while ( i < j ){
```

```
        while ( i < j && R[j].key >= x.key ) j--;
```

```
        if ( i < j ){
```

```
            R[i].key = R[j].key; i++;
```

```
        }
```

```
        while ( i < j && R[i].key <= x.key ) i++;
```

```
        if ( i < j ){
```

```
            R[j].key = R[i].key; j--;
```

```
        }
```

```
    }
```

```
    R[i].key = x.key;
```

```
    return i;
```

```
}
```

(1) 设 $\text{RecType } R[] = \{52, 14, 256, -9, -38, 30, 128, 258, 64\}$, 给出执行 $\text{f33}(R, 0, 8)$ 后 R 的结果。

(2) 给出该算法的功能。

34. 34. 已知二叉树的存储结构类型定义如下:

```
typedef struct node {
```

```
    int data;
```

```
    struct node *lchild, *rchild;
```

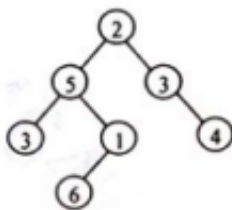
```
} BinNode;
```

```
typedef BinNode *BinTree;
```

编写递归算法, 对于给定的一棵二叉树 T , 计算并返回所有结点 data 域的值之和。

函数原型为: $\text{int f34}(\text{BinTree } T)$;。例如, 对于题 34 图所示的二叉树 T , $\text{f34}(T)$ 应

返回 24。



题 34 图

答案

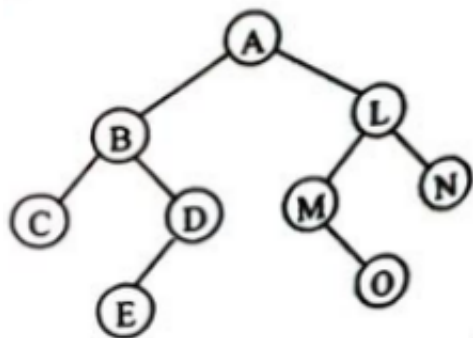
- 1.B
- 2.A
- 3.D
- 4.B
- 5.B
- 6.C
- 7.C
- 8.C
- 9.D
- 10.D
- 11.D
- 12.C
- 13.D
- 14.B
- 15.C



16.
答：顺序存储结构
17.
答：SXSSXSXX
18.
答：递归表
19.
答：(c,d),c)
20.
答：线索
21.
答： $2n-1$
22.
答： $n-1$
23.
答： $k_1 \leq k_2$ 且 $k_i \leq k_{2i+1} (1 \leq i \leq [n/2])$
24.
答：分块查找
25.
答：2
26.
答：

	i	j	v
0	1	2	-8
1	1	4	6
2	2	1	7
3	3	1	-5
4	3	5	3

27.
答：



28.
答：

(1) G 的邻接矩阵

$$\begin{bmatrix} \infty & 5 & 3 & \infty & 2 \\ \infty & \infty & \infty & 6 & \infty \\ \infty & 1 & \infty & 2 & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & 6 & 10 & 4 & \infty \end{bmatrix}$$

(2) 根据迪杰斯特拉算法求得的各条最短路径依次是

次序	最短路径	长度
1	AE	2
2	AC	3
3	ACB	4
4	ACD	5

29.
答：

29. (1) 散列表及各关键字的探查次数

(4分)

下标(地址)	0	1	2	3	4	5	6	7	8	9	10
散列表	53	23		91	26	15	72	7	52	31	65
探查次数	3	1		1	1	2	1	1	1	1	1

【评分说明】散列表的内容部分，共给3分，探查次数部分，共给1分。部分正确酌情给分。

$$(2) ASL = (3+1+1+1+2+1+1+1+1+1+1)/10 = 1.3$$

(1分)

30.
答：

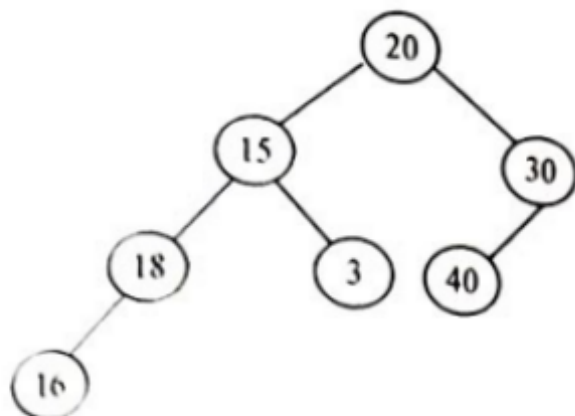
30. (1) SL1->data 中的数据是 (52, 14, 256, 6, -9, 81, 128, 257, 64) (2分)

SL2->data 中的数据是 (84, 28, 193, 15, 29, 51, 144, 265) (2分)

(2) 该算法比较两个线性表中相同下标位置的两个元素(以较短者为限)，将两者之和放到原来较小者的位置，若原来两者相等，则将两者之和放到 SL2 中。 (1分)

31.
答：

(1)



(4分)

(2) 函数 f31() 的功能是：将二叉树 T 的每个结点的高度较高的子树放到左子树位置，高度较矮的子树放到右子树位置。

(1分)

32.

答：(1) high = mid - 1

(2) mid

(3) i >= inspace

33.

答：(1) R 中的值是 {30, 14, -38, -9, 52, 256, 128, 258, 64}

(2) 该算法选择第一个元素作为基准，实现对数组的一次划分。(或实现快速排序的一次划分。)

34.

答：

34. 参考程序如下。

```
int f34 ( BinTree T )
{   int leftSum = 0, rightSum = 0;
    if ( T == NULL ) return 0;
    else {
        if ( T->lchild!=NULL ) leftSum = f34( T->lchild );
        if ( T->rchild!=NULL ) rightSum = f34( T->rchild );
        return leftSum + rightSum + T->data;
    }
}
```